



GUÍA COMPLETA – IDEA B: Agente de Triage de Incidentes

AI Agentic Hackathon: Construye más con MaaS

Organizado por Huawei Cloud y Kostra

Fecha: 3 de Septiembre de 2026

Tiempo efectivo de desarrollo: 5.5 horas



TABLA DE CONTENIDOS

1. [Visión General del Proyecto](#)
 2. [Fase 0: Setup Inicial](#)
 3. [Fase 1: Configuración y Cliente LLM](#)
 4. [Fase 2: Base de Datos](#)
 5. [Fase 3: Herramientas del Agente](#)
 6. [Fase 4: Agente con LangGraph](#)
 7. [Fase 5: UI en Streamlit](#)
 8. [Fase 6: Testing y Validación](#)
 9. [Fase 7: Pre-Hackathon Checklist](#)
 10. [Fase 8: Checklist Onboarding](#)
 11. [Fase 9: Cronograma del Hackathon](#)
 12. [Fase 10: Estructura del Pitch](#)
 13. [Resumen y Próximos Pasos](#)
-



VISIÓN GENERAL

El Problema

Equipos de operaciones gastan **30-60 minutos** en cada incidente operacional para:

- Clasificar el tipo de incidente
- Identificar el componente afectado
- Buscar soluciones conocidas
- Estimar tiempo de resolución

La Solución

Un **Agente Autónomo de IA** que ejecuta este diagnóstico en **<15 segundos** mediante:

1. **Análisis:** Clasificación automática del incidente
2. **Diagnóstico:** Ejecución de 6 herramientas en paralelo
3. **Recomendación:** Risk scoring, checklist, escalation

Tecnología

- **Orquestación:** LangGraph (razonamiento agéntico visible)
 - **LLM:** Huawei Cloud MaaS (Pangu 40B) + fallback OpenAI
 - **UI:** Streamlit (desarrollo rápido)
 - **DB:** SQLite (portable, sin dependencias)
-

FASE 0: SETUP INICIAL (15 MINUTOS)

Paso 1: Crear el repositorio y estructura base

```
# Crear carpeta del proyecto
mkdir ai-hackathon-triage-agent
cd ai-hackathon-triage-agent

# Inicializar git y python environment
git init
python -m venv venv
source venv/bin/activate # En Windows: venv\Scripts\activate

# Crear estructura de carpetas
mkdir -p src/{agent,tools,db,ui} data tests notebooks
```

```
# Crear archivos iniciales
touch .env.example .gitignore requirements.txt README.md
touch src/__init__.py src/config.py src/llm_client.py
touch src/agent/{__init__.py,state.py,graph.py,nodes.py,tools.py}
touch src/tools/{__init__.py,validators.py,queries.py,analyzers.py}
touch src/db/{__init__.py,schema.sql,fixtures.py,client.py}
touch src/ui/{__init__.py,app.py}
touch tests/{__init__.py,test_agent.py,test_tools.py}
```

Paso 2: requirements.txt

```
# LLM & Orchestration
langgraph==0.0.68
langchain==0.1.20
langchain-community==0.0.20
python-dotenv==1.0.0

# Web & API
httpx==0.25.0
requests==2.31.0

# UI
streamlit==1.28.0
streamlit-aggrid==0.3.4

# Data & Validation
pydantic==2.5.0
pandas==1.5.0
numpy==1.24.0
sqlalchemy==2.0.23

# Utils
typing-extensions==4.8.0
jsonschema==4.20.0

# Testing
pytest==7.4.0
pytest-asyncio==0.21.0
```

Instalar dependencias:

```
pip install -r requirements.txt
```

Paso 3: .env.example

```
# .env.example
# LLM Configuration
LLM_PROVIDER=openai # Cambiar a 'huawei' en el hackathon
OPENAI_API_KEY=sk-...
OPENAI_MODEL=gpt-4
OPENAI_BASE_URL=https://api.openai.com/v1

# Huawei Cloud MaaS (llenar en onboarding)
HUAWEI_API_KEY=
HUAWEI_MODEL=pangu-40b
HUAWEI_REGION=cn-north-4

# Database
DB_PATH=data/incidents.db

# App
LOG_LEVEL=INFO
DEBUG=False

cp .env.example .env
```

FASE 1: CONFIGURACIÓN Y CLIENTE LLM (30 MINUTOS)

Paso 4: src/config.py

```
# src/config.py
import os
from pathlib import Path
from typing import Literal
from dotenv import load_dotenv

load_dotenv()

class Config:
    # Paths
    PROJECT_ROOT = Path(__file__).parent.parent
    DATA_DIR = PROJECT_ROOT / "data"
    DB_PATH = DATA_DIR / "incidents.db"

    # Ensure data dir exists
    DATA_DIR.mkdir(exist_ok=True)
```

```

# LLM
LLM_PROVIDER: Literal["openai", "huawei"] = os.getenv("LLM_PROVIDER", "openai")

# OpenAI
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY", "")
OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-4")
OPENAI_BASE_URL = os.getenv("OPENAI_BASE_URL", "https://api.openai.com/v1")

# Huawei Cloud MaaS
HUAWEI_API_KEY = os.getenv("HUAWEI_API_KEY", "")
HUAWEI_MODEL = os.getenv("HUAWEI_MODEL", "pangu-40b")
HUAWEI_REGION = os.getenv("HUAWEI_REGION", "cn-north-4")
HUAWEI_BASE_URL = f"https://maas.cn-north-4.myhuaweicloud.com" # Confirmar e

# App
DEBUG = os.getenv("DEBUG", "False").lower() == "true"
LOG_LEVEL = os.getenv("LOG_LEVEL", "INFO")

# Agent Parameters
AGENT_TEMPERATURE = 0.3 # Bajo para decisiones deterministas
AGENT_MAX_TOKENS = 2000
AGENT_TIMEOUT_SECONDS = 30

config = Config()

```

Paso 5: src/llm_client.py (CRÍTICO)

```

# src/llm_client.py
import os
import json
import asyncio
from abc import ABC, abstractmethod
from typing import Optional
from datetime import datetime
import httpx

from src.config import config

class LLMClient(ABC):
    """Abstract base para clientes LLM."""

    @abstractmethod
    async def call(
        self,
        messages: list[dict],
        temperature: Optional[float] = None,

```

```

        max_tokens: Optional[int] = None,
    ) -> str:
        """Llamar al LLM y devolver texto de respuesta."""
        pass

class OpenAICompatibleClient(LLMClient):
    """Cliente compatible con OpenAI API (OpenAI, Azure, MaaS compatible)."""

    def __init__(
        self,
        base_url: str,
        api_key: str,
        model: str,
        timeout: int = 30,
    ):
        self.base_url = base_url.rstrip("/")
        self.api_key = api_key
        self.model = model
        self.timeout = timeout
        self.client = httpx.AsyncClient(timeout=timeout)

    async def call(
        self,
        messages: list[dict],
        temperature: Optional[float] = None,
        max_tokens: Optional[int] = None,
    ) -> str:
        """Llamar OpenAI-compatible API."""
        temperature = temperature or config.AGENT_TEMPERATURE
        max_tokens = max_tokens or config.AGENT_MAX_TOKENS

        payload = {
            "model": self.model,
            "messages": messages,
            "temperature": temperature,
            "max_tokens": max_tokens,
        }

        headers = {
            "Authorization": f"Bearer {self.api_key}",
            "Content-Type": "application/json",
        }

        try:
            response = await self.client.post(
                f"{self.base_url}/v1/chat/completions",

```

```

        json=payload,
        headers=headers,
    )
    response.raise_for_status()
    data = response.json()
    return data["choices"][0]["message"]["content"]
except httpx.TimeoutException:
    raise RuntimeError(f"LLM request timeout after {self.timeout}s")
except httpx.HTTPError as e:
    raise RuntimeError(f"LLM API error: {e}")

```

```
class HuaweiCloudMaaSClient(LLMClient):
```

```
    """Cliente nativo para Huawei Cloud MaaS."""
```

```

    def __init__(
        self,
        api_key: str,
        model: str,
        region: str = "cn-north-4",
        timeout: int = 30,
    ):
        self.api_key = api_key
        self.model = model
        self.region = region
        self.timeout = timeout
        self.base_url = f"https://maas.{region}.myhuaweicloud.com"
        self.client = httpx.AsyncClient(timeout=timeout)

```

```

    async def call(
        self,
        messages: list[dict],
        temperature: Optional[float] = None,
        max_tokens: Optional[int] = None,
    ) -> str:

```

```
    """
```

```
    Llamar Huawei Cloud MaaS API.
```

```
    NOTA: Estos detalles se confirman en onboarding (01/Sept).
```

```
    Este skeleton es compatible con OpenAI API, pero Huawei podría tener
    diferencias en auth headers, estructura de request, etc.
```

```
    """
```

```
    temperature = temperature or config.AGENT_TEMPERATURE
```

```
    max_tokens = max_tokens or config.AGENT_MAX_TOKENS
```

```
    # Estructura asume compatibilidad OpenAI; ajustar según docs Huawei
```

```
    payload = {
```

```

        "model": self.model,
        "messages": messages,
        "temperature": temperature,
        "max_tokens": max_tokens,
    }

    # Huawei IAM: podría ser JWT, OAuth2, etc.
    # Placeholder: confirmar en onboarding
    headers = {
        "Authorization": f"Bearer {self.api_key}",
        "Content-Type": "application/json",
    }

    try:
        response = await self.client.post(
            f"{self.base_url}/v1/chat/completions",
            json=payload,
            headers=headers,
        )
        response.raise_for_status()
        data = response.json()
        # Estructura de respuesta: confirmar con docs Huawei
        return data["choices"][0]["message"]["content"]
    except httpx.TimeoutException:
        raise RuntimeError(f"Huawei MaaS timeout after {self.timeout}s")
    except httpx.HTTPError as e:
        raise RuntimeError(f"Huawei MaaS error: {e}")


def get_llm_client() -> LLMClient:
    """Factory: devuelve el cliente LLM según configuración."""

    if config.LLM_PROVIDER == "huawei":
        if not config.HUAWEI_API_KEY:
            raise ValueError(
                "HUAWEI_API_KEY no configurada. "
                "Completa .env con credenciales del onboarding."
            )
        return HuaweiCloudMaaSClient(
            api_key=config.HUAWEI_API_KEY,
            model=config.HUAWEI_MODEL,
            region=config.HUAWEI_REGION,
        )
    else: # openai (default)
        if not config.OPENAI_API_KEY:
            raise ValueError(
                "OPENAI_API_KEY no configurada. "

```



```

        "Completa .env o usa `openai.api_key = '...'`"
    )
    return OpenAICompatibleClient(
        base_url=config.OPENAI_BASE_URL,
        api_key=config.OPENAI_API_KEY,
        model=config.OPENAI_MODEL,
    )

# Para usos síncronos en Streamlit
_client_cache: Optional[LLMClient] = None

def get_llm_client_sync() -> LLMClient:
    """Get cached LLM client (for Streamlit)."""
    global _client_cache
    if _client_cache is None:
        _client_cache = get_llm_client()
    return _client_cache

```

FASE 2: BASE DE DATOS (15 MINUTOS)

Paso 6: src/db/schema.sql

```

-- src/db/schema.sql
-- Base de datos de incidentes históricos para queries del agente

CREATE TABLE IF NOT EXISTS incident_history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    incident_type TEXT NOT NULL,
    component TEXT NOT NULL,
    severity TEXT CHECK(severity IN ('P1', 'P2', 'P3', 'P4')),
    mttc_minutes FLOAT, -- Mean Time To Detect
    mtrr_minutes FLOAT, -- Mean Time To Recover
    resolution TEXT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS health_checks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    component TEXT NOT NULL,
    status TEXT CHECK(status IN ('UP', 'DEGRADED', 'DOWN')),
    latency_ms FLOAT,
    last_check DATETIME DEFAULT CURRENT_TIMESTAMP
);

```

```
);

CREATE TABLE IF NOT EXISTS knowledge_base (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    incident_type TEXT NOT NULL,
    component TEXT,
    symptom TEXT,
    root_cause TEXT,
    resolution_steps TEXT, -- JSON array
    confidence FLOAT DEFAULT 0.8
);

CREATE INDEX idx_incident_type ON incident_history(incident_type);
CREATE INDEX idx_component ON incident_history(component);
CREATE INDEX idx_severity ON incident_history(severity);
```

Paso 7: src/db/client.py

```
# src/db/client.py
import sqlite3
from pathlib import Path
from typing import List, Dict, Optional, Any
import json
from datetime import datetime

from src.config import config

class DBClient:
    """Cliente SQLite para queries ágénticas."""

    def __init__(self, db_path: Optional[Path] = None):
        self.db_path = db_path or config.DB_PATH
        self._ensure_db()

    def _ensure_db(self):
        """Crear DB y schema si no existen."""
        if not self.db_path.exists():
            self.db_path.parent.mkdir(parents=True, exist_ok=True)
            schema_path = Path(__file__).parent / "schema.sql"
            with open(schema_path, "r") as f:
                schema = f.read()
            conn = sqlite3.connect(self.db_path)
            conn.executescript(schema)
            conn.commit()
            conn.close()
```

```

def _get_connection(self) -> sqlite3.Connection:
    conn = sqlite3.connect(self.db_path)
    conn.row_factory = sqlite3.Row
    return conn

def query_incident_history(
    self,
    incident_type: Optional[str] = None,
    component: Optional[str] = None,
    severity: Optional[str] = None,
    limit: int = 10,
) -> List[Dict[str, Any]]:
    """Query histórico de incidentes para análisis de patrones."""
    conn = self._get_connection()
    cursor = conn.cursor()

    query = "SELECT * FROM incident_history WHERE 1=1"
    params = []

    if incident_type:
        query += " AND incident_type LIKE ?"
        params.append(f"%{incident_type}%")
    if component:
        query += " AND component LIKE ?"
        params.append(f"%{component}%")
    if severity:
        query += " AND severity = ?"
        params.append(severity)

    query += f" ORDER BY timestamp DESC LIMIT {limit}"

    cursor.execute(query, params)
    rows = cursor.fetchall()
    conn.close()

    return [dict(row) for row in rows]

def get_component_status(self, component: str) -> Optional[Dict[str, Any]]:
    """Obtener status actual de un componente."""
    conn = self._get_connection()
    cursor = conn.cursor()

    cursor.execute(
        "SELECT * FROM health_checks WHERE component = ? "
        "ORDER BY last_check DESC LIMIT 1",
        (component,)
    )

```

```

        row = cursor.fetchone()
        conn.close()

    return dict(row) if row else None

def search_knowledge_base(
    self,
    incident_type: Optional[str] = None,
    component: Optional[str] = None,
    symptom: Optional[str] = None,
) -> List[Dict[str, Any]]:
    """Buscar en Knowledge Base soluciones conocidas."""
    conn = self._get_connection()
    cursor = conn.cursor()

    query = "SELECT * FROM knowledge_base WHERE 1=1"
    params = []

    if incident_type:
        query += " AND incident_type LIKE ?"
        params.append(f"%{incident_type}%")
    if component:
        query += " AND component LIKE ?"
        params.append(f"%{component}%")
    if symptom:
        query += " AND symptom LIKE ?"
        params.append(f"%{symptom}%")

    query += " ORDER BY confidence DESC"

    cursor.execute(query, params)
    rows = cursor.fetchall()
    conn.close()

    results = []
    for row in rows:
        row_dict = dict(row)
        # Parse resolution_steps JSON
        if row_dict.get("resolution_steps"):
            try:
                row_dict["resolution_steps"] = json.loads(row_dict["resolutio
            except:
                pass
        results.append(row_dict)

    return results

```

```
# Singleton global
_db_client: Optional[DBClient] = None

def get_db() -> DBClient:
    global _db_client
    if _db_client is None:
        _db_client = DBClient()
    return _db_client
```

Paso 8: src/db/fixtures.py (Datos de Prueba)

```
# src/db/fixtures.py
"""Seed data para desarrollo y testing."""
import sqlite3
from pathlib import Path
from src.db.client import get_db
import json

def seed_knowledge_base():
    """Cargar Knowledge Base con soluciones comunes."""
    db = get_db()
    conn = sqlite3.connect(db.db_path)
    cursor = conn.cursor()

    kb_solutions = [
        {
            "incident_type": "Database Connectivity",
            "component": "PostgreSQL",
            "symptom": "Connection refused on port 5432",
            "root_cause": "Database service down or network unreachable",
            "resolution_steps": json.dumps([
                "1. Check if DB pod is running: kubectl get pods -l app=postgres",
                "2. Check DB logs: kubectl logs <postgres-pod>",
                "3. Verify network policy allows traffic from app to DB",
                "4. Restart DB if needed: kubectl rollout restart deployment/postgres"
            ]),
            "confidence": 0.95,
        },
        {
            "incident_type": "Memory Leak",
            "component": "Application",
            "symptom": "Memory usage increases over time, OOM kills",
            "root_cause": "Unreleased object references in cache or event listeners",
            "resolution_steps": json.dumps([
                "1. Check memory profile: node --inspect=0.0.0.0:9229 app.js",
            ]),
        }
    ]

    cursor.executemany(
        """
        INSERT OR REPLACE INTO knowledge_base (
            incident_type, component, symptom, root_cause, resolution_steps, confidence
        ) VALUES (
            ?, ?, ?, ?, ?, ?
        )
        """,
        kb_solutions
    )
    conn.commit()
    conn.close()
```

```

        "2. Use Chrome DevTools Memory profiler to find leaking objects",
        "3. Review recent changes to caching logic",
        "4. Implement memory limits in cache (e.g., maxsize=10000)",
    ]),
    "confidence": 0.88,
},
{
    "incident_type": "High Latency",
    "component": "API Gateway",
    "symptom": "Response times >5s, P95 latency spikes",
    "root_cause": "Upstream service slow, load imbalance, or resource con",
    "resolution_steps": json.dumps([
        "1. Check upstream service health: curl -s http://service/health",
        "2. Monitor CPU/Memory: top, docker stats",
        "3. Review recent deployments: git log --oneline",
        "4. Increase replicas if CPU >80%: kubectl scale deployment api-g",
    ]),
    "confidence": 0.82,
},
{
    "incident_type": "Deployment Failed",
    "component": "CI/CD Pipeline",
    "symptom": "Deployment stuck in pending, rollout timeout",
    "root_cause": "Image pull error, insufficient resources, or liveness",
    "resolution_steps": json.dumps([
        "1. Check pod status: kubectl describe pod <pod>",
        "2. Check image availability: docker pull <image>",
        "3. Check cluster resources: kubectl top nodes",
        "4. Review probe config and adjust timeouts if necessary",
    ]),
    "confidence": 0.90,
},
]

```

```

for solution in kb_solutions:

```

```

    cursor.execute(
        """INSERT INTO knowledge_base
        (incident_type, component, symptom, root_cause, resolution_steps,
        VALUES (?, ?, ?, ?, ?, ?))""",
        (
            solution["incident_type"],
            solution["component"],
            solution["symptom"],
            solution["root_cause"],
            solution["resolution_steps"],
            solution["confidence"],
        ),
    ),

```

```

    )

    conn.commit()
    conn.close()
    print("✅ Knowledge Base seeded")

def seed_incident_history():
    """Cargar histórico de incidentes para análisis de patrones."""
    db = get_db()
    conn = sqlite3.connect(db.db_path)
    cursor = conn.cursor()

    incidents = [
        ("Database Connectivity", "PostgreSQL", "P1", 5.0, 45.0),
        ("High Latency", "API Gateway", "P2", 10.0, 30.0),
        ("Memory Leak", "Application", "P1", 20.0, 120.0),
        ("Deployment Failed", "CI/CD Pipeline", "P2", 15.0, 60.0),
        ("Database Connectivity", "PostgreSQL", "P1", 3.0, 38.0),
        ("High Latency", "API Gateway", "P3", 8.0, 25.0),
    ]

    for inc_type, component, severity, mttdd, mttr in incidents:
        cursor.execute(
            """INSERT INTO incident_history
               (incident_type, component, severity, mttdd_minutes, mttr_minutes)
               VALUES (?, ?, ?, ?, ?)""",
            (inc_type, component, severity, mttdd, mttr),
        )

    conn.commit()
    conn.close()
    print("✅ Incident history seeded")

def seed_health_checks():
    """Cargar status de componentes."""
    db = get_db()
    conn = sqlite3.connect(db.db_path)
    cursor = conn.cursor()

    components = [
        ("PostgreSQL", "UP", 15.0),
        ("API Gateway", "DEGRADED", 450.0),
        ("Application", "UP", 120.0),
        ("Redis Cache", "UP", 5.0),
        ("Message Queue", "DOWN", None),
    ]

```

```

]

for component, status, latency in components:
    cursor.execute(
        """INSERT INTO health_checks (component, status, latency_ms)
           VALUES (?, ?, ?)""",
        (component, status, latency),
    )

conn.commit()
conn.close()
print("✅ Health checks seeded")


def reset_and_seed():
    """Resetear DB y cargar fixtures."""
    db = get_db()

    # Drop all tables
    conn = sqlite3.connect(db.db_path)
    cursor = conn.cursor()
    cursor.execute("DROP TABLE IF EXISTS incident_history")
    cursor.execute("DROP TABLE IF EXISTS health_checks")
    cursor.execute("DROP TABLE IF EXISTS knowledge_base")
    conn.commit()
    conn.close()

    # Recreate schema
    db._ensure_db()

    # Seed data
    seed_knowledge_base()
    seed_incident_history()
    seed_health_checks()

    print("✅ Database reset and seeded successfully")


if __name__ == "__main__":
    reset_and_seed()

```

Ejecutar seed:

```
python -m src.db.fixtures
```

FASE 3: HERRAMIENTAS DEL AGENTE (20 MINUTOS)

Paso 9: src/agent/state.py

```
# src/agent/state.py
"""Estado del agente (TypedDict para LangGraph)."""
from typing import TypedDict, Optional, List, Dict, Any
from datetime import datetime

class IncidentData(TypedDict, total=False):
    """Datos del incidente reportado."""
    description: str
    component: Optional[str]
    severity: Optional[str] # P1, P2, P3, P4
    timestamp: datetime

class DiagnosticStep(TypedDict, total=False):
    """Un paso de diagnóstico ejecutado."""
    step_number: int
    tool_name: str
    input: Dict[str, Any]
    output: str
    reasoning: str

class AgentState(TypedDict, total=False):
    """Estado del agente de triage."""
    # Input
    incident: IncidentData

    # Reasoning & Analysis
    initial_analysis: str # Análisis inicial del agente
    identified_type: Optional[str] # Tipo de incidente clasificado
    identified_component: Optional[str] # Componente afectado

    # Diagnostic Steps
    diagnostic_steps: List[DiagnosticStep] # Histórico de herramientas ejecutadas
    component_status: Optional[Dict[str, Any]] # Status del componente
    historical_patterns: List[Dict[str, Any]] # Incidentes similares pasados
    kb_solutions: List[Dict[str, Any]] # Soluciones conocidas

    # Analysis & Recommendations
    root_cause_hypothesis: str
    risk_score: float # 0-10
    estimated_mttr_minutes: Optional[float]
```

```

escalation_required: bool
escalation_path: Optional[str] # Team/contact

# Diagnostics Checklist
diagnostics_checklist: List[str] # Pasos que el usuario debe ejecutar

# Final Output
final_recommendation: str # Resumen ejecutivo

# Metadata
messages: List[dict] # Chat history para contexto

```

Paso 10: src/tools/validators.py

```

# src/tools/validators.py
"""Tool 1: Validar y parsear descripción del incidente."""
from typing import Dict, Any, Optional
import re
from datetime import datetime

def validate_incident_description(description: str) -> Dict[str, Any]:
    """
    Validar y extraer información estructurada de descripción textual.

    Simula una herramienta que parsea logs, alertas, etc.
    En producción, esto sería OCR de eventos, parsing de logs, etc.
    """

    output = {
        "is_valid": True,
        "raw_text": description,
        "extracted_fields": {},
        "confidence_score": 0.8,
        "warnings": [],
    }

    # Buscar keywords de componentes
    components = [
        "database", "postgres", "mysql", "redis",
        "api", "gateway", "load balancer",
        "application", "app server", "node",
        "message queue", "kafka", "rabbitmq",
        "cache", "memcached",
        "network", "dns", "cdn",
        "ci/cd", "pipeline", "deployment",
    ]

```

```

component_matches = [c for c in components if c.lower() in description.lower()]
if component_matches:
    output["extracted_fields"]["component"] = component_matches[0].title()

# Buscar keywords de severidad
if any(word in description.lower() for word in ["critical", "down", "outage",
    output["extracted_fields"]["severity"] = "P1"
elif any(word in description.lower() for word in ["error", "significant", "p2
    output["extracted_fields"]["severity"] = "P2"
elif any(word in description.lower() for word in ["warning", "slow", "p3"]):
    output["extracted_fields"]["severity"] = "P3"
else:
    output["extracted_fields"]["severity"] = "P4"

# Buscar keywords de tipo de incidente
incident_types = {
    "connectivity": ["connection", "refused", "timeout", "unreachable"],
    "performance": ["slow", "latency", "lag", "timeout"],
    "memory": ["oom", "memory leak", "heap", "memory"],
    "crash": ["crash", "segfault", "panic", "core dump"],
    "deployment": ["deploy", "rollout", "failed", "unable to start"],
    "data": ["data corruption", "missing data", "inconsistent"],
}

for inc_type, keywords in incident_types.items():
    if any(kw in description.lower() for kw in keywords):
        output["extracted_fields"]["incident_type"] = inc_type.title()
        break

# Validaciones
if len(description.strip()) < 10:
    output["is_valid"] = False
    output["warnings"].append("Descripción muy corta (<10 caracteres)")

if not output["extracted_fields"].get("component"):
    output["warnings"].append("No se identificó componente específico")

output["extracted_fields"]["timestamp"] = datetime.now().isoformat()

return output

def extract_error_codes(description: str) -> Dict[str, Any]:
    """Extraer códigos de error (HTTP, system, etc.)."""

    # Buscar códigos HTTP

```

```

http_codes = re.findall(r'\b(4\d{2}|5\d{2})\b', description)

# Buscar códigos de error Unix
error_codes = re.findall(r'\berr[no]?[: ]?(\d+)\b', description, re.IGNORECAS

return {
    "http_codes": http_codes or None,
    "error_codes": error_codes or None,
    "has_stack_trace": "traceback" in description.lower() or "stack" in descr
}

```

Paso 11: src/tools/queries.py

```

# src/tools/queries.py
"""Tool 2: Query histórico de incidentes y status de componentes."""
from typing import Dict, Any, List, Optional
from src.db.client import get_db

def query_historical_incidents(component: str, incident_type: Optional[str] = None)
    """
    Query histórico de incidentes similares.
    Herramienta que el agente usa para encontrar patrones.
    """

    db = get_db()

    # Query histórico
    incidents = db.query_incident_history(
        component=component,
        incident_type=incident_type,
        limit=5,
    )

    # Calcular estadísticas
    if incidents:
        avg_mttr = sum(i["mttr_minutes"] for i in incidents if i["mttr_minutes"])
        avg_mtttd = sum(i["mttd_minutes"] for i in incidents if i["mttd_minutes"])
    else:
        avg_mttr = None
        avg_mtttd = None

    return {
        "component": component,
        "total_incidents": len(incidents),
        "incidents": incidents,
        "average_mttr_minutes": avg_mttr,
    }

```

```

        "average_mttdd_minutes": avg_mttdd,
        "most_common_severity": max(
            set(i["severity"] for i in incidents),
            key=lambda x: sum(1 for i in incidents if i["severity"] == x),
            default=None
        ) if incidents else None,
    }

def check_component_health(component: str) -> Dict[str, Any]:
    """
    Chequear status actual de un componente.
    Simula un healthcheck real (HTTP, TCP, etc.)
    """

    db = get_db()
    status = db.get_component_status(component)

    if not status:
        return {
            "component": component,
            "status": "UNKNOWN",
            "message": f"No health data found for {component}",
        }

    # Interpretar latencia
    latency = status.get("latency_ms")
    if latency is None:
        latency_status = "NO_DATA"
    elif latency > 1000:
        latency_status = "CRITICAL"
    elif latency > 500:
        latency_status = "WARNING"
    else:
        latency_status = "OK"

    return {
        "component": component,
        "status": status["status"],
        "latency_ms": latency,
        "latency_status": latency_status,
        "last_check": status.get("last_check"),
        "is_operational": status["status"] == "UP",
    }

def search_solutions_in_kb(

```

```

        incident_type: str,
        component: str,
        symptom: Optional[str] = None,
) -> Dict[str, Any]:
    """
    Buscar soluciones conocidas en Knowledge Base.
    Herramienta que el agente usa para encontrar remediaciones comunes.
    """

    db = get_db()
    solutions = db.search_knowledge_base(
        incident_type=incident_type,
        component=component,
        symptom=symptom,
    )

    return {
        "incident_type": incident_type,
        "component": component,
        "solutions_found": len(solutions),
        "solutions": solutions,
        "best_match": solutions[0] if solutions else None,
    }

```

Paso 12: src/tools/analyzers.py

```

# src/tools/analyzers.py
"""Tool 3: Análisis custom y cálculo de scores."""
from typing import Dict, Any, List
from datetime import datetime
import math

def calculate_risk_score(
    component: str,
    severity: str,
    is_operational: bool,
    historical_mttd: float = 10.0,
) -> Dict[str, Any]:
    """
    Calcular risk score (0-10) basado en múltiples factores.
    Herramienta para cuantificar criticidad.
    """

    score = 0.0

    # Factor 1: Severidad declarada

```

```

severity_map = {"P1": 3.0, "P2": 2.0, "P3": 1.5, "P4": 1.0}
score += severity_map.get(severity, 1.0)

# Factor 2: Status operacional
if not is_operational:
    score += 3.0
elif "database" in component.lower() or "queue" in component.lower():
    score += 2.0 # Componentes críticos más puntaje

# Factor 3: Frecuencia histórica (basado en MTTR)
if historical_mttr < 5:
    score += 2.0
elif historical_mttr < 15:
    score += 1.0

# Normalizar a 0-10
final_score = min(10.0, score)

# Categoría
if final_score >= 8:
    category = "CRITICAL"
elif final_score >= 6:
    category = "HIGH"
elif final_score >= 4:
    category = "MEDIUM"
else:
    category = "LOW"

return {
    "risk_score": round(final_score, 1),
    "category": category,
    "factors": {
        "severity_component": severity_map.get(severity, 1.0),
        "operational_status": 0.0 if is_operational else 3.0,
        "historical_frequency": 2.0 if historical_mttr < 5 else (1.0 if histo
    },
    "recommendation": "Escalate immediately" if final_score >= 8 else "Monito
}

def estimate_sla_and_mttr(
    severity: str,
    historical_mttr: Optional[float] = None,
) -> Dict[str, Any]:
    """
    Estimar SLA y MTTR esperado.
    """

```

```

sla_minutes = {
    "P1": {"response": 15, "resolution": 60},
    "P2": {"response": 30, "resolution": 240},
    "P3": {"response": 60, "resolution": 480},
    "P4": {"response": 120, "resolution": 2880},
}

sla = sla_minutes.get(severity, sla_minutes["P4"])

estimated_mttr = historical_mttr or sla["resolution"]

# Calcular SLA compliance risk
if estimated_mttr > sla["resolution"]:
    sla_risk = "HIGH"
elif estimated_mttr > sla["resolution"] * 0.8:
    sla_risk = "MEDIUM"
else:
    sla_risk = "LOW"

return {
    "severity": severity,
    "sla_response_minutes": sla["response"],
    "sla_resolution_minutes": sla["resolution"],
    "estimated_mttr_minutes": estimated_mttr,
    "sla_risk": sla_risk,
}

def generate_diagnostics_checklist(
    incident_type: str,
    component: str,
    solutions: List[Dict[str, Any]] = None,
) -> Dict[str, Any]:
    """
    Generar checklist de diagnóstico para el humano.
    """

    checklist = []

    # Pasos genéricos
    checklist.append(f"1. Verify {component} is running and accessible")
    checklist.append(f"2. Check logs for {component}: tail -f /var/log/{component}")
    checklist.append(f"3. Monitor resource usage: top, docker stats, kubectl top")

    # Pasos específicos según tipo
    if "connectivity" in incident_type.lower():

```



```

        checklist.extend([
            "4. Test network reachability: ping, telnet, curl",
            "5. Check firewall rules: iptables -L",
            "6. Verify DNS resolution: nslookup, dig",
        ])
    elif "performance" in incident_type.lower():
        checklist.extend([
            "4. Profile application: perf, strace, dtrace",
            "5. Check database query performance: EXPLAIN ANALYZE",
            "6. Review recent code deployments",
        ])
    elif "memory" in incident_type.lower():
        checklist.extend([
            "4. Analyze memory dump",
            "5. Check for memory leaks: valgrind, node inspect",
            "6. Review cache configuration and limits",
        ])

    # Escalation
    checklist.append("7. If above steps don't resolve, escalate to platform team")

    return {
        "incident_type": incident_type,
        "component": component,
        "checklist": checklist,
        "estimated_time_minutes": 15,
        "kb_solutions_count": len(solutions or []),
    }

```

FASE 4: AGENTE CON LANGGRAPH (25 MINUTOS)

Paso 13: src/agent/tools.py

```

# src/agent/tools.py
"""LangGraph tool definitions para el agente."""
from typing import Any
from langchain_core.tools import tool

from src.tools.validators import validate_incident_description
from src.tools.queries import (
    query_historical_incidents,
    check_component_health,
    search_solutions_in_kb,

```

```

)
from src.tools.analyzers import (
    calculate_risk_score,
    estimate_sla_and_mttr,
    generate_diagnostics_checklist,
)

@tool
def tool_validate_incident(description: str) -> dict:
    """
    Validar descripción del incidente y extraer campos estructurados.
    Simula parsing de alerts, logs, reportes de usuario.
    """
    return validate_incident_description(description)

@tool
def tool_query_historical(component: str, incident_type: str = None) -> dict:
    """
    Buscar incidentes históricos similares para identificar patrones.
    Incluye MTTR promedio, severidad más común, etc.
    """
    return query_historical_incidents(component, incident_type)

@tool
def tool_check_health(component: str) -> dict:
    """
    Chequear status actual de un componente.
    Retorna: operacional, latencia, last check time.
    """
    return check_component_health(component)

@tool
def tool_search_kb(incident_type: str, component: str, symptom: str = None) -> dict:
    """
    Buscar soluciones conocidas en Knowledge Base.
    Retorna lista de soluciones paso a paso con confidence scores.
    """
    return search_solutions_in_kb(incident_type, component, symptom)

@tool
def tool_calculate_risk(
    component: str,

```

```

        severity: str,
        is_operational: bool,
        historical_mttd: float = 10.0,
    ) -> dict:
        """
        Calcular risk score (0-10) para el incidente.
        Usado para determinar escalation y prioridad.
        """
        return calculate_risk_score(component, severity, is_operational, historical_m

```

@tool

```

def tool_estimate_sla(severity: str, historical_mttr: float = None) -> dict:
    """
    Estimar SLA y MTTR esperado basado en histórico.
    Retorna: SLA targets, estimated resolution time, risk level.
    """
    return estimate_sla_and_mttr(severity, historical_mttr)

```

@tool

```

def tool_generate_checklist(
    incident_type: str,
    component: str,
    solutions_count: int = 0,
) -> dict:
    """
    Generar diagnostics checklist para el operador humano.
    Pasos específicos según tipo de incidente.
    """
    solutions = [{}] * solutions_count # Dummy para contar
    return generate_diagnostics_checklist(incident_type, component, solutions)

```

Mapeo para LangGraph

```

TOOLS_MAP = {
    "validate_incident": tool_validate_incident,
    "query_historical": tool_query_historical,
    "check_health": tool_check_health,
    "search_kb": tool_search_kb,
    "calculate_risk": tool_calculate_risk,
    "estimate_sla": tool_estimate_sla,
    "generate_checklist": tool_generate_checklist,
}

```

```

TOOLS_LIST = list(TOOLS_MAP.values())

```

Paso 14: src/agent/nodes.py

```
# src/agent/nodes.py
"""Nodos del grafo agéntico (LangGraph)."""
import json
from datetime import datetime
from typing import Any
import asyncio

from src.agent.state import AgentState, DiagnosticStep, IncidentData
from src.llm_client import get_llm_client_sync
from src.agent.tools import TOOLS_LIST, TOOLS_MAP
from src.config import config

def node_analyze_incident(state: AgentState) -> dict:
    """
    Nodo 1: Análisis inicial del incidente.
    El agente lee la descripción y genera análisis de alto nivel.
    """

    incident = state.get("incident", {})
    description = incident.get("description", "")

    if not description:
        return {
            "initial_analysis": "ERROR: No incident description provided",
            "messages": state.get("messages", []),
        }

    # Usar el cliente LLM para análisis inicial
    client = get_llm_client_sync()

    system_prompt = """You are an expert incident responder and triage agent.
    Analyze the incident description and provide:
    1. Initial classification (type of incident)
    2. Likely affected component
    3. Preliminary severity assessment
    4. What information you need to gather next

    Be concise and actionable."""

    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": f"Incident description: {description}"},
    ]
```

```

try:
    # Correr async call en sync context
    loop = asyncio.new_event_loop()
    asyncio.set_event_loop(loop)
    analysis = loop.run_until_complete(client.call(messages))
    loop.close()
except Exception as e:
    analysis = f"Error during analysis: {str(e)}"

return {
    "initial_analysis": analysis,
    "messages": messages + [{"role": "assistant", "content": analysis}],
}

def node_execute_tools(state: AgentState) -> dict:
    """
    Nodo 2: Ejecutar herramientas de diagnóstico.
    El agente decide qué herramientas usar basado en el análisis previo.
    """

    incident = state.get("incident", {})
    initial_analysis = state.get("initial_analysis", "")
    component = incident.get("component") or state.get("identified_component", "")
    severity = incident.get("severity") or state.get("identified_type", "P2")

    diagnostic_steps = state.get("diagnostic_steps", [])

    # Tool 1: Validar descripción
    validation_result = TOOLS_MAP["validate_incident"](
        incident.get("description", "")
    )
    diagnostic_steps.append(DiagnosticStep(
        step_number=1,
        tool_name="validate_incident",
        input={"description": incident.get("description", "")},
        output=json.dumps(validation_result),
        reasoning="Extract structured fields from incident description",
    ))

    # Actualizar component y severity si se extrajeron
    if not component and validation_result.get("extracted_fields", {}).get("component"):
        component = validation_result["extracted_fields"]["component"]
    if not severity and validation_result.get("extracted_fields", {}).get("severity"):
        severity = validation_result["extracted_fields"]["severity"]

```

```

# Tool 2: Chequear health del componente
if component:
    health_result = TOOLS_MAP["check_health"](component)
    diagnostic_steps.append(DiagnosticStep(
        step_number=2,
        tool_name="check_health",
        input={"component": component},
        output=json.dumps(health_result),
        reasoning=f"Verify {component} operational status",
    ))
    component_status = health_result
else:
    component_status = None

# Tool 3: Query histórico
incident_type = validation_result.get("extracted_fields", {}).get("incident_t
if component:
    history_result = TOOLS_MAP["query_historical"](component, incident_type)
    diagnostic_steps.append(DiagnosticStep(
        step_number=3,
        tool_name="query_historical",
        input={"component": component, "incident_type": incident_type},
        output=json.dumps(history_result),
        reasoning="Find patterns from historical incidents",
    ))
    historical_patterns = history_result.get("incidents", [])
    historical_mtttd = history_result.get("average_mtttd_minutes", 10.0)
else:
    historical_patterns = []
    historical_mtttd = 10.0

return {
    "diagnostic_steps": diagnostic_steps,
    "component_status": component_status,
    "historical_patterns": historical_patterns,
    "identified_component": component,
    "identified_type": incident_type,
}

def node_search_solutions(state: AgentState) -> dict:
    """
    Nodo 3: Buscar soluciones conocidas en KB.
    """

    incident_type = state.get("identified_type", "Unknown")
    component = state.get("identified_component", "")

```

```

description = state.get("incident", {}).get("description", "")

diagnostic_steps = state.get("diagnostic_steps", [])

# Tool 4: Search KB
kb_result = TOOLS_MAP["search_kb"](incident_type, component, description)
diagnostic_steps.append(DiagnosticStep(
    step_number=4,
    tool_name="search_kb",
    input={"incident_type": incident_type, "component": component},
    output=json.dumps({
        "solutions_found": kb_result.get("solutions_found", 0),
        "best_match": kb_result.get("best_match"),
    }),
    reasoning="Find known solutions from knowledge base",
))

kb_solutions = kb_result.get("solutions", [])

return {
    "kb_solutions": kb_solutions,
    "diagnostic_steps": diagnostic_steps,
}

def node_calculate_score(state: AgentState) -> dict:
    """
    Nodo 4: Calcular risk score y SLA.
    """

    component = state.get("identified_component", "")
    severity = state.get("incident", {}).get("severity", "P2")
    component_status = state.get("component_status", {})
    is_operational = component_status.get("is_operational", True)
    historical_patterns = state.get("historical_patterns", [])

    diagnostic_steps = state.get("diagnostic_steps", [])

    # Tool 5: Calculate risk
    historical_mtttd = 10.0
    if historical_patterns:
        total = sum(p.get("mttd_minutes", 10.0) for p in historical_patterns)
        historical_mtttd = total / len(historical_patterns)

    risk_result = TOOLS_MAP["calculate_risk"](
        component, severity, is_operational, historical_mtttd
    )

```

```

diagnostic_steps.append(DiagnosticStep(
    step_number=5,
    tool_name="calculate_risk",
    input={
        "component": component,
        "severity": severity,
        "is_operational": is_operational,
        "historical_mttd": historical_mttd,
    },
    output=json.dumps(risk_result),
    reasoning="Quantify incident severity and risk",
))

# Tool 6: Estimate SLA
sla_result = TOOLS_MAP["estimate_sla"](severity, historical_mttd)
diagnostic_steps.append(DiagnosticStep(
    step_number=6,
    tool_name="estimate_sla",
    input={"severity": severity, "historical_mttr": historical_mttd},
    output=json.dumps(sla_result),
    reasoning="Calculate SLA targets and MTTR estimates",
))

escalation_required = risk_result.get("risk_score", 0) >= 7
escalation_path = "On-call Platform Engineer" if escalation_required else Non

return {
    "risk_score": risk_result.get("risk_score", 5.0),
    "estimated_mttr_minutes": sla_result.get("estimated_mttr_minutes"),
    "escalation_required": escalation_required,
    "escalation_path": escalation_path,
    "diagnostic_steps": diagnostic_steps,
}

def node_generate_output(state: AgentState) -> dict:
    """
    Nodo 5: Generar output final y recomendaciones.
    """

    component = state.get("identified_component", "Unknown")
    incident_type = state.get("identified_type", "Unknown")
    kb_solutions = state.get("kb_solutions", [])
    risk_score = state.get("risk_score", 5.0)
    severity = state.get("incident", {}).get("severity", "P2")
    escalation_required = state.get("escalation_required", False)

```



```

diagnostic_steps = state.get("diagnostic_steps", [])

# Tool 7: Generate checklist
checklist_result = TOOLS_MAP["generate_checklist"](
    incident_type, component, len(kb_solutions)
)
diagnostic_steps.append(DiagnosticStep(
    step_number=7,
    tool_name="generate_checklist",
    input={
        "incident_type": incident_type,
        "component": component,
        "solutions_count": len(kb_solutions),
    },
    output=json.dumps(checklist_result),
    reasoning="Create actionable diagnostics checklist",
))

diagnostics_checklist = checklist_result.get("checklist", [])

# Generar recomendación final con LLM
client = get_llm_client_sync()

system_prompt = """You are an incident triage expert.
Synthesize all diagnostic data and provide a concise, actionable summary.
Include: root cause hypothesis, recommended next steps, escalation decision.
Keep to 3-4 sentences."""

diagnostic_summary = "\n".join([
    f"Step {s['step_number']}: {s['tool_name']} - {s['reasoning']}",
    f"Result: {s['output'][:200]}..."
] for s in diagnostic_steps[:3])

messages = [
    {"role": "system", "content": system_prompt},
    {
        "role": "user",
        "content": f"""Incident Summary:
Type: {incident_type}
Component: {component}
Risk Score: {risk_score}/10
Severity: {severity}
Escalation Required: {escalation_required}

Diagnostics performed:
{diagnostic_summary}

```

```

KB Solutions found: {len(kb_solutions)}
"""
    },
]

try:
    loop = asyncio.new_event_loop()
    asyncio.set_event_loop(loop)
    final_rec = loop.run_until_complete(client.call(messages))
    loop.close()
except Exception as e:
    final_rec = f"Escalate to {state.get('escalation_path', 'support team')}}"

root_cause = "Analyzed: " + (kb_solutions[0].get("root_cause", "Unknown") if

return {
    "root_cause_hypothesis": root_cause,
    "final_recommendation": final_rec,
    "diagnostics_checklist": diagnostics_checklist,
    "diagnostic_steps": diagnostic_steps,
}

```

Paso 15: src/agent/graph.py

```

# src/agent/graph.py
"""Definición del grafo agéntico con LangGraph."""
from langgraph.graph import StateGraph, END
from src.agent.state import AgentState
from src.agent.nodes import (
    node_analyze_incident,
    node_execute_tools,
    node_search_solutions,
    node_calculate_score,
    node_generate_output,
)

def create_triage_graph():
    """Crear el grafo de triage."""

    graph = StateGraph(AgentState)

    # Añadir nodos
    graph.add_node("analyze", node_analyze_incident)
    graph.add_node("tools", node_execute_tools)
    graph.add_node("kb_search", node_search_solutions)

```

```

graph.add_node("scoring", node_calculate_score)
graph.add_node("output", node_generate_output)

# Definir flujo
graph.set_entry_point("analyze")
graph.add_edge("analyze", "tools")
graph.add_edge("tools", "kb_search")
graph.add_edge("kb_search", "scoring")
graph.add_edge("scoring", "output")
graph.add_edge("output", END)

# Compilar
return graph.compile()

# Singleton
_triage_graph = None

def get_triage_graph():
    """Obtener la instancia del grafo (cached)."""
    global _triage_graph
    if _triage_graph is None:
        _triage_graph = create_triage_graph()
    return _triage_graph

```

FASE 5: UI EN STREAMLIT (15 MINUTOS)

Paso 16: src/ui/app.py

```

# src/ui/app.py
"""Streamlit UI para el agente de triage."""
import streamlit as st
from datetime import datetime
import json

from src.agent.graph import get_triage_graph
from src.agent.state import AgentState, IncidentData

# Configuración Streamlit
st.set_page_config(
    page_title="AI Incident Triage Agent",
    page_icon="🚨",
    layout="wide",

```

```

        initial_sidebar_state="expanded",
    )

# CSS personalizado
st.markdown("""
<style>
    .stTabs [data-baseweb="tab-list"] button {
        font-size: 16px;
        font-weight: bold;
    }
    .metric-card {
        background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
        padding: 20px;
        border-radius: 10px;
        color: white;
        margin: 10px 0;
    }
</style>
""", unsafe_allow_html=True)

def main():
    st.title("🚨 AI Incident Triage Agent")
    st.markdown("**Powered by Huawei Cloud MaaS + LangGraph**")

    # Sidebar: Configuración
    with st.sidebar:
        st.header("⚙️ Configuration")

        llm_provider = st.selectbox(
            "LLM Provider",
            ["OpenAI (Default)", "Huawei Cloud MaaS"],
            help="Switch between providers for testing",
        )

        st.markdown("---")
        st.subheader("🇮🇹 About this Agent")
        st.info(
            """
            **Triage Agent Flow:**
            1. Analyze incident description
            2. Execute diagnostic tools
            3. Search knowledge base
            4. Calculate risk & SLA
            5. Generate recommendations
            """
        )

```

```

# Main content
st.header("📄 Report an Incident")

# Input form
col1, col2 = st.columns([2, 1])

with col1:
    description = st.text_area(
        "Incident Description",
        placeholder="E.g., API Gateway returning 503 errors, response time >5s",
        height=150,
        help="Describe the issue in detail",
    )

with col2:
    st.markdown("### Quick Select")
    quick_select = st.selectbox(
        "Or choose a sample:",
        [
            "Custom...",
            "🔴 Database Connectivity Down",
            "🟡 High Latency Spike",
            "🟡 Memory Leak Detected",
            "🟡 Deployment Failed",
        ],
    )

    if quick_select != "Custom...":
        samples = {
            "🔴 Database Connectivity Down":
                "PostgreSQL connection refused on port 5432. Application cannot connect.",
            "🟡 High Latency Spike":
                "API Gateway response times increased from 100ms to 5000ms. P99 latency is high.",
            "🟡 Memory Leak Detected":
                "Node.js application process memory increasing over time. RSS is high.",
            "🟡 Deployment Failed":
                "Latest deployment stuck in pending. ImagePullBackOff error in logs."
        }
        description = samples[quick_select]

# Run analysis button
if st.button("🔍 Run Triage Analysis", key="run_analysis", use_container_width=True):
    if not description.strip():
        st.error("Please provide an incident description")
        return

    with st.spinner("🔄 Analyzing incident..."):

```

```

# Crear estado inicial
initial_state: AgentState = {
    "incident": IncidentData(
        description=description,
        timestamp=datetime.now(),
    ),
    "messages": [],
}

# Ejecutar grafo
try:
    graph = get_triage_graph()
    final_state = graph.invoke(initial_state)
except Exception as e:
    st.error(f"Error during analysis: {str(e)}")
    st.stop()

# Mostrar resultados
st.success("✅ Analysis complete!")

# Tabs para resultados
tab1, tab2, tab3, tab4 = st.tabs(
    [
        📊 "Summary",
        🛠️ "Diagnostics",
        📖 "Solutions",
        📋 "Checklist"
    ]
)

with tab1:
    col1, col2, col3, col4 = st.columns(4)

    with col1:
        st.metric(
            "Risk Score",
            f"{final_state.get('risk_score', 0):.1f}/10",
            delta=None,
        )

    with col2:
        severity = final_state.get("incident", {}).get("severity", "P2")
        st.metric("Severity", severity)

    with col3:
        mttr = final_state.get("estimated_mttr_minutes")
        st.metric(
            "Estimated MTTR",
            f"{mttr:.0f} min" if mttr else "TBD",
        )

    with col4:

```

```

        escalation = "🔴 YES" if final_state.get("escalation_required") else
        st.metric("Escalation Required", escalation)

    st.divider()

    st.subheader("Analysis Summary")
    st.write(final_state.get("initial_analysis", "No analysis available"))

    st.divider()

    st.subheader("Root Cause Hypothesis")
    st.info(final_state.get("root_cause_hypothesis", "Needs investigation"))

    st.subheader("Recommendation")
    st.success(final_state.get("final_recommendation", "Escalate to support"))

with tab2:
    st.subheader("🔧 Diagnostic Steps Executed")

    diagnostic_steps = final_state.get("diagnostic_steps", [])

    for step in diagnostic_steps:
        with st.expander(
            f"Step {step.get('step_number')}: {step.get('tool_name')} "
            f"({step.get('reasoning')})",
            expanded=step.get("step_number") == 1,
        ):
            st.caption(f"Reasoning: {step.get('reasoning')}")

            col1, col2 = st.columns(2)
            with col1:
                st.write("**Input:**")
                st.json(step.get("input", {}))

            with col2:
                st.write("**Output:**")
                try:
                    output = json.loads(step.get("output", "{}"))
                    st.json(output)
                except:
                    st.text(step.get("output", ""))

with tab3:
    st.subheader("📖 Knowledge Base Solutions")

    kb_solutions = final_state.get("kb_solutions", [])

```

```

if kb_solutions:
    for i, solution in enumerate(kb_solutions, 1):
        with st.expander(
            f"Solution {i}: {solution.get('incident_type')} "
            f"(Confidence: {solution.get('confidence', 0.8):.0%})",
            expanded=i == 1,
        ):
            st.write(f"***Root Cause:** {solution.get('root_cause')}")
            st.write("***Resolution Steps:**")

            try:
                steps = json.loads(solution.get("resolution_steps", ""))
                for step in steps:
                    st.write(f"- {step}")
            except:
                st.text(solution.get("resolution_steps", "N/A"))
        else:
            st.info("No KB solutions found. Escalate to expert team.")

with tab4:
    st.subheader("📋 Diagnostics Checklist")

    checklist = final_state.get("diagnostics_checklist", [])

    if checklist:
        for item in checklist:
            st.write(f"☐ {item}")
    else:
        st.info("No checklist generated")

if __name__ == "__main__":
    main()

```

Ejecutar la app:

```
streamlit run src/ui/app.py
```

FASE 6: TESTING Y VALIDACIÓN (15 MINUTOS)

Paso 17: tests/test_tools.py

```
# tests/test_tools.py
```



```

"""Tests para las herramientas."""
import pytest
from src.tools.validators import validate_incident_description
from src.tools.queries import (
    query_historical_incidents,
    check_component_health,
)
from src.tools.analyzers import (
    calculate_risk_score,
    estimate_sla_and_mttr,
)

def test_validate_incident_valid():
    """Test validar descripción válida."""
    desc = "PostgreSQL connection refused on port 5432. Database unreachable."
    result = validate_incident_description(desc)

    assert result["is_valid"] == True
    assert "component" in result["extracted_fields"]
    assert result["extracted_fields"]["component"] == "Postgres"

def test_validate_incident_severity():
    """Test detectar severidad."""
    desc = "Critical: All API servers down. System outage."
    result = validate_incident_description(desc)

    assert result["extracted_fields"]["severity"] == "P1"

def test_check_component_health():
    """Test chequear health de componente."""
    result = check_component_health("PostgreSQL")

    assert "component" in result
    assert "status" in result
    assert result["status"] in ["UP", "DEGRADED", "DOWN", "UNKNOWN"]

def test_calculate_risk_critical():
    """Test calcular risk score."""
    result = calculate_risk_score(
        component="PostgreSQL",
        severity="P1",
        is_operational=False,
        historical_mttd=5.0,
    )

```

```

    )

    assert result["risk_score"] >= 8.0
    assert result["category"] == "CRITICAL"

def test_estimate_sla_p1():
    """Test estimar SLA para P1."""
    result = estimate_sla_and_mttr("P1", 45.0)

    assert result["sla_response_minutes"] == 15
    assert result["sla_resolution_minutes"] == 60
    assert result["estimated_mttr_minutes"] == 45.0

if __name__ == "__main__":
    pytest.main([__file__, "-v"])

```

Ejecutar tests:

```
pytest tests/ -v
```

FASE 7: PRE-HACKATHON CHECKLIST

Paso 18: README.md (Documentación)

```
# AI Incident Triage Agent
```

```
Agente autónomo que clasifica y diagnostica incidentes operacionales en <30 segun
```

```
## Arquitectura
```

User Input (Incident Description) ↓ [LangGraph Agentic Loop]

- Node 1: Analyze (LLM classification)
- Node 2: Execute Tools (validation, healthchecks, queries)
- Node 3: KB Search (find known solutions)
- Node 4: Score & SLA (risk calculation)

- Node 5: Output (generate recommendations) ↓ Structured Output (risk score, checklist, escalation)

```
## Quick Start

```bash
1. Setup
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt

2. Seed database
python -m src.db.fixtures

3. Run UI
streamlit run src/ui/app.py
```

Luego abre `http://localhost:8501`

## Configuration

Editar `.env`:

```
LLM_PROVIDER=openai # Cambiar a 'huawei' en hackathon
OPENAI_API_KEY=sk-...
OPENAI_MODEL=gpt-4
```

## Tools Available

1. **tool\_validate\_incident** - Parse structured fields from description
2. **tool\_query\_historical** - Find similar past incidents
3. **tool\_check\_health** - Current component status
4. **tool\_search\_kb** - Known solutions from knowledge base
5. **tool\_calculate\_risk** - Risk score (0-10)
6. **tool\_estimate\_sla** - SLA targets and MTTR
7. **tool\_generate\_checklist** - Diagnostics steps for operator

# Demo Incidents

Pre-cargadas en Streamlit UI:

- Database Connectivity Down
- High Latency Spike
- Memory Leak Detected
- Deployment Failed

## Testing

```
pytest tests/ -v
```

## Performance

- Analysis time: ~5-10 seconds (LLM dependent)
- Works offline with fixtures
- Fallback to OpenAI if Huawei MaaS unavailable

---

## Hackathon: AI Agentic Hackathon - Sept 3, 2026

---

## FASE 8: CHECKLIST ONBOARDING

### Preguntas CRÍTICAS para Huawei Cloud MaaS (Sesión 01/Sept, 12:00 hrs)

#### 1. Authentication & Access

- [ ] ¿Qué tipo de authentication usa MaaS? (JWT, OAuth2, API Key simple?)
- [ ] ¿Cómo obtener API Key? (Console, CLI, email token?)
- [ ] ¿Cuál es la estructura del header de autenticación?
- [ ] ¿Hay restricciones de región? (cn-north-4, cn-east-3, etc.)
- [ ] ¿Cómo manejar rate limiting? (HTTP 429, X-RateLimit headers?)

#### 2. API Endpoint & Protocol

- [ ] ¿Cuál es el base URL correcto?
- [ ] ¿Compatible con OpenAI API spec? (POST /v1/chat/completions)

- [ ] Si NO es OpenAI compatible, ¿cuál es la estructura exacta?
  - Request payload format
  - Response structure
  - Error response format
- [ ] ¿Soporta streaming? (Server-Sent Events, WebSocket?)
- [ ] Timeout máximo recomendado?

#### #### 3. Models Available

- [ ] ¿Qué modelos están disponibles?
  - Pangu-40B
  - DeepSeek
  - Llama variants
  - Others?
- [ ] ¿Cuál es el model string correcto para cada uno?
- [ ] ¿Hay diferencias en parámetros soportados?
  - temperature range
  - max\_tokens limits
  - top\_p, top\_k support?
- [ ] ¿Cuál model es RECOMENDADO para razonamiento/tool use?

#### #### 4. Tool Use / Function Calling

- [ ] ¿Soporta function calling (OpenAI style)?
- [ ] Si sí, ¿cuál es el formato de definición de funciones?
- [ ] ¿Hay límites en cantidad o tamaño de funciones definidas?
- [ ] ¿Cómo se devuelven los resultados de funciones?

#### #### 5. Cost & Quotas

- [ ] ¿Pricing por token? (input vs output differentiation?)
- [ ] ¿Hay free tier / credits para hackathon?
- [ ] ¿Rate limits? (requests/sec, tokens/min?)
- [ ] ¿Cómo monitorear usage en tiempo real?

#### #### 6. Reliability & SLAs

- [ ] ¿SLA de uptime?
- [ ] ¿Hay fallback/redundancy?
- [ ] ¿Tiempo máximo aceptable para request timeout?
- [ ] ¿Hay un status page o incident channel?

#### #### 7. Specifics for Hackathon

- [ ] ¿Credenciales pre-provisionadas para participantes?
- [ ] ¿Hay sandbox/staging environment?
- [ ] ¿Support contact durante el hackathon?
- [ ] ¿Hay ejemplos de código (Python, Node.js)?
- [ ] ¿Documentación live (URL)?

**\*\*Después del Onboarding:\*\***

1. Actualizar `.env` con credenciales

2. Ajustar `src/llm\_client.py` si necesario
3. Test connection: `python -c "from src.llm\_client import get\_llm\_client; c = ge`
4. Run full demo con Streamlit

---

## ## FASE 9: CRONOGRAMA DEL HACKATHON (3 SEPTIEMBRE)

### ### 09:00-09:30 | Coffee + Networking

#### \*\*Checklist Técnico:\*\*

- [ ] Git repo clonado y actualizado
- [ ] `.env` configurado con credenciales
- [ ] `streamlit run` ejecuta sin errores
- [ ] DB seeded: `python -m src.db.fixtures`
- [ ] Test query a MaaS (o OpenAI fallback)

### ### 09:30-09:45 | Kickoff

- Escuchar brief del hackathon
- Identificar jueces y criterios
- Networking rápido

### ### 09:45-13:00 | BUILD BLOQUE 1 (3h 15 min)

#### \*\*09:45-10:15 | Setup y Architecture Review (30 min)\*\*

- Confirmar stack funcionando
- Commit inicial: `git commit -m "Init: project structure"`
- \*\*Tareas en paralelo:\*\*
  - Persona A: Refinar prompts del agente
  - Persona B: Ampliar knowledge base (más soluciones)
  - Persona C: Diseño UI (Streamlit themes)

#### \*\*10:15-11:30 | Core Development (75 min)\*\*

- \*\*Persona A:\*\* Mejorar nodos del grafo
  - Refinar `node\_analyze\_incident()`
  - Refinar `node\_generate\_output()`
  - Test con incidentes complejos
- \*\*Persona B:\*\* Ampliar herramientas
  - Agregar más soluciones a KB
  - Mejorar lógica de `tool\_calculate\_risk()`
  - Crear datos de prueba más realistas
- \*\*Persona C:\*\* Pulir UI
  - Agregar visualizaciones
  - Mejorar UX/diseño
  - Crear landing page

**\*\*11:30-12:30 | Integration & Testing (60 min)\*\***

- Integración de cambios
- `pytest tests/ -v` - todos los tests pasan
- Test end-to-end con 5-10 incidentes
- **\*\*Medir latencia:\*\*** < 15 segundos?

**\*\*12:30-13:00 | Polish & Documentation (30 min)\*\***

- Código limpio
- Git commits bien estructurados
- README actualizado
- **\*\*Grabar BACKUP VIDEO\*\*** de demo manual

### 13:00-14:00 | LUNCH

### 14:00-17:00 | BUILD BLOQUE 2 (3h)

**\*\*14:00-15:00 | Bug Fixes & Optimizations (60 min)\*\***

- Revisión de issues
- Priorizar errores críticos
- Timeout/latency fixes

**\*\*15:00-16:00 | Feature Polish & Demo Readiness (60 min)\*\***

- Casos de uso "wow"
- Script de demo escrito
- Laptop de demostración preparada:
  - Streamlit app abierta
  - Credenciales confirmadas
  - Cache limpio

**\*\*16:00-16:30 | BACKUP VIDEO RECORDING (30 min)\*\***

- **\*\*CRÍTICO:\*\*** Grabar video de 3 minutos
  - Incidente → Análisis → Output
  - Sin edición, video directo
  - Plan B si MaaS falla en vivo

**\*\*16:30-17:00 | Code Freeze & Pitch Prep (30 min)\*\***

- **\*\*16:30:\*\*** Freeze absoluto de código
  - `git tag v1.0-submission`
- **\*\*16:30-17:00:\*\*** Ensayar pitch
  - Timing exacto (3 min)
  - Delivery confident
  - Slides listas

### 17:00 | CODE FREEZE

-  No más código

### 17:10-17:20 | PITCHES EN VIVO

- 3 minutos por equipo
- Demo en vivo
- Q&A jueces

### 17:20-18:30 | WINNERS + NETWORKING

---

### Contingency Plan

**\*\*Si MaaS falla:\*\***

```bash

sed -i 's/LLM_PROVIDER=huawei/LLM_PROVIDER=openai/' .env

streamlit run src/ui/app.py

Si network falla: Demo de video pregrabado

Si Streamlit falla: Ejecutar agent directamente via CLI

Si grafo falla: Fallback a salida JSON estructurada

FASE 10: ESTRUCTURA DEL PITCH (3 MINUTOS)

[0:00-0:30] PROBLEMA

Guion:

"Equipos de operaciones gastan 30-60 minutos clasificando y diagnosticando cada incidente. Ese tiempo es crítico cuando los minutos cuentan.

Del 100% de faltas operacionales, el 60% sigue patrones predecibles. Pero se resuelven igual de lento cada vez."

[0:30-1:30] SOLUCIÓN AGÉNTICA

Guion:

"Nuestro AI Triage Agent resuelve esto con un ciclo de razonamiento automático:

1 Analyze - El agente lee la descripción y clasifica tipo + componente

2 Diagnose - Ejecuta 6 herramientas en paralelo:

- Chequea status actual
- Busca incidentes similares

- Consulta Knowledge Base

Recommend - Risk score, MTTR, checklist

Todo en <15 segundos.

Rol de Huawei Cloud MaaS: Usa modelos fundacionales (Pangu 40B) para razonamiento. MaaS permite escalabilidad sin overhead de fine-tuning."

[1:30-2:45] DEMO EN VIVO (1 min 15 seg)

Script:

1. Ingresa incidente
2. Click "Run Analysis"
3. Muestra Summary, Diagnostics, Solutions, Checklist
4. Si falla: "Aquí está el video pregrabado..."

[2:45-3:00] IMPACTO (15 seg)

Guion:

"Con este agente:

- MTDD de 30 min → <1 min
- 60% de incidentes resueltos automáticamente
- 30 horas ahorradas/mes por datacenter

Escalable gracias a Huawei Cloud MaaS."

RESUMEN EJECUCIÓN COMPLETA

| Fase | Duración | Completado |
|-----------------|----------|---|
| 0. Setup | 15 min |  |
| 1. Config & LLM | 30 min |  |
| 2. DB | 15 min |  |
| 3. Tools | 20 min |  |
| | | |

| | | |
|----------------------------|-----------------|---|
| 4. Agent (LangGraph) | 25 min | ✓ |
| 5. UI (Streamlit) | 15 min | ✓ |
| 6. Testing | 15 min | ✓ |
| 7. Pre-Hackathon Checklist | 5 min | ✓ |
| 8. Onboarding | 15 preguntas | ✓ |
| 9. Cronograma | Minuto-a-minuto | ✓ |
| 10. Pitch | 3 min structure | ✓ |

PRÓXIMOS PASOS INMEDIATOS

Hoy mismo:

1. Clona y instala:

```
pip install -r requirements.txt
python -m src.db.fixtures
streamlit run src/ui/app.py
```

2. Prueba con incidente de muestra

3. Confirma MaaS: Actualiza `.env` y test:

```
python -c "from src.llm_client import get_llm_client; c = get_llm_client(); pr
```

Sept 1 (Onboarding):

- 15 preguntas a Huawei
- Ajusta `HuaweiCloudMaaSClient`
- Test final de conexión

Sept 2 (Pre-hackathon):

- Última pasada de código
- Practica pitch 2x

- Prepara laptop

Sept 3 (Hackathon):

- Sigue cronograma
 - Sé agile: feature → test → demo
 - Demo < 15 seg en vivo, video de respaldo
-

SOPORTE

Si tienes dudas:

1. Revisa el README.md
 2. Chequea los tests: `pytest tests/ -v`
 3. Consulta en el onboarding de Huawei (01/Sept)
 4. Usa fallback a OpenAI si MaaS falla
-

¡Buena suerte en el hackathon! 🚀